MATERIAL DE ESTUDO

STREAMING DE DADOS PARA A WEB

Conceitos, Arquitetura e Exemplos Práticos em Java & Python

Java | Python | HTTP | WebSocket | SSE | gRPC | Reactive

Agenda

01 Introdução ao Streaming

02 HTTP Chunked Transfer Encoding

03 Long Polling

04 Server-Sent Events (SSE)

05 WebSockets

06 gRPC Streaming

07 Reactive Streams

08 Comparação & Produção

Introdução — O que é Streaming?

Request-Response Tradicional

- 1. Cliente envia requisição
- 2. Servidor processa completamente
- 3. Servidor retorna resposta inteira
- 4. Conexão é encerrada

❌ Alta latência percebida
❌ Espera pelo processamento total
❌ Alto consumo de memória

Streaming de Dados

- 1. Cliente abre conexão
- 2. Servidor envia dados em fragmentos
- 3. Cliente processa progressivamente
- 4. Conexão persiste durante o fluxo

✓ Baixa latência percebida
✓ Processamento incremental
✓ Eficiência de memória

Modelos de Comunicação — Visão Geral

Modelo	Direção	Conexão	Latência	Caso de Uso
Request-Response	Unidirecional	Por requisição	Alta	APIs REST
Long Polling	Srv → Client	Mantida até resposta	Média	Notificações simples
Server-Sent Events	Srv → Client	Persistente	Baixa	Feeds, dashboards
WebSocket	Bidirecional	Persistente	Muito baixa	Chat, jogos
gRPC Streaming	Bidirecional	HTTP/2 multiplex	Muito baixa	Microserviços, IoT

HTTP Chunked Transfer Encoding

RFC 7230 | Transfer-Encoding: chunked

Permite que o servidor inicie a transmissão antes de conhecer o tamanho total da resposta.
O corpo é dividido em fragmentos (chunks), cada um precedido por seu tamanho em hexadecimal.

HTTP Wire Format

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: text/plain

7\r\n
Mozilla\r\n
9\r\n
Developer\r\n
0\r\n        ← fim do stream
\r\n
```

Quando Utilizar

Exportação de grandes arquivos

Sem buffering total em memória

Relatórios paginados

Cliente processa linha a linha

Respostas de LLMs

Texto progressivo (streaming de tokens)

Dumps de banco de dados

Payloads de vários GBs

Chunked Streaming — Java (Spring Boot)

Java — StreamingResponseBody

```java
@GetMapping(value = "/export", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<StreamingResponseBody> streamExport() {

    StreamingResponseBody stream = outputStream -> {
        try (PrintWriter writer = new PrintWriter(
                new BufferedWriter(new OutputStreamWriter(outputStream)))) {

            writer.print("[");
            writer.flush();

            for (int i = 0; i < 5000; i++) {
                if (i > 0) writer.print(",");
                writer.printf({"id":%d,"valor":%.2f}, i, Math.random() * 1000);
                writer.flush();  // envia o chunk imediatamente
            }
            writer.print("]");
            writer.flush();
        }
    };
    return ResponseEntity.ok().body(stream);
}
```

✦ *writer.flush() após cada escrita força o envio imediato do chunk ao cliente*

Chunked Streaming — Python (Flask / FastAPI)

Python — Flask Generator

```python
from flask import Flask, Response
import json, time

app = Flask(__name__)

def gerar_registros(total: int):
    """Gerador: cada yield = um chunk"""
    for i in range(total):
        yield json.dumps({
            "id": i, "valor": i ** 2
        }) + "\n"
        time.sleep(0.005)

@app.route("/api/export")
def stream_export():
    return Response(
        gerar_registros(10_000),
        mimetype="application/x-ndjson",
        headers={"X-Accel-Buffering": "no"}
    )
```

JavaScript — Consumo (Fetch API)

```javascript
async function consumir(url) {
  const resp = await fetch(url);
  const reader = resp.body.getReader();
  const dec = new TextDecoder();
  let buf = "";

  while (true) {
    const {done, value} = await reader.read();
    if (done) break;

    buf += dec.decode(value, {stream: true});
    const linhas = buf.split("\n");
    buf = linhas.pop();

    for (const linha of linhas) {
      const obj = JSON.parse(linha);
      console.log("Recebido:", obj);
    }
  }
}
consumir("/api/export");
```

⚠ *Nginx: adicionar proxy_buffering off; para que chunks cheguem imediatamente ao cliente*

Long Polling

Simulando tempo real sobre HTTP convencional

Long Polling — Ciclo de Vida

1. Cliente envia GET /events?lastId=42
2. Servidor verifica: há eventos novos?
3. Se SIM → retorna imediatamente com dados
4. Se NÃO → mantém requisição suspensa (hold)
5. Ao receber evento ou timeout → responde
6. Cliente processa e reconecta imediatamente

✓ Compatível com HTTP/1.0, proxies e firewalls ⚠ Overhead de reconexão constante ✓ Não requer suporte especial no servidor

Long Polling — Implementação

Java — Spring WebFlux (Reativo)

```java
@RestController
public class LongPollController {

  private final BlockingQueue<String> queue =
      new LinkedBlockingQueue<>();

  @GetMapping("/api/longpoll/events")
  public Mono<ResponseEntity<String>> poll() {
    return Mono
      .fromCallable(() -> queue.poll(30,
TimeUnit.SECONDS))
      .subscribeOn(Schedulers.boundedElastic())
      .map(event -> event != null
        ? ResponseEntity.ok(event)
        : ResponseEntity.<String>noContent().build()
      );
  }

  @PostMapping("/api/longpoll/publish")
  public ResponseEntity<Void> publish(
                        @RequestBody String payload) {
    queue.offer(payload);
    return ResponseEntity.accepted().build();
  }
```

Python — FastAPI Assíncrono

```python
from fastapi import FastAPI
from fastapi.responses import JSONResponse
import asyncio

app = FastAPI()
event_queue: asyncio.Queue = asyncio.Queue()

@app.get("/api/longpoll/events")
async def poll(timeout: int = 30):
    try:
        evento = await asyncio.wait_for(
            event_queue.get(),
            timeout=timeout
        )
        return JSONResponse({"evento": evento})
    except asyncio.TimeoutError:
        return JSONResponse(None, status_code=204)

@app.post("/api/longpoll/publish")
async def publish(payload: dict):
    await event_queue.put(payload)
    return {"status": "publicado"}
```

Server-Sent Events (SSE)

Padrão W3C para streaming unidirecional sobre HTTP

SSE — Formato do Protocolo

SSE Wire Format (text/event-stream)

```
id: 42
event: temperatura
data: {"sensor":"S01","graus":23.5}
retry: 3000

id: 43
event: alerta
data: {"mensagem":"Temperatura elevada!"}

: ping  ← comentário (mantém conexão)
```

🔄 Reconexão Automática — o browser reconecta automaticamente ao detectar queda da conexão, enviando Last-Event-ID com o ID do último evento processado.

`id:`

Rastreia último evento recebido. Reenviado via Last-Event-ID na reconexão.

`event:`

Tipo do evento. O cliente registra listeners por tipo.

`data:`

Carga útil (texto ou JSON). Múltiplas linhas são concatenadas.

`retry:`

Intervalo (ms) que o cliente aguarda antes de reconectar.

`:`

Comentário. Mantém a conexão TCP ativa através de proxies.

SSE — Java Spring WebFlux

Java — Flux<ServerSentEvent> com multicast

```
@RestController
public class SseController {

  // Multicast sink: todos os subscribers recebem o mesmo evento
  private final Sinks.Many<String> sink =
      Sinks.many().multicast().onBackpressureBuffer();

  @GetMapping(value = "/sse/notificacoes",
              produces = MediaType.TEXT_EVENT_STREAM_VALUE)
  public Flux<ServerSentEvent<String>> stream() {
    return sink.asFlux()
        .map(msg -> ServerSentEvent.<String>builder()
            .id(String.valueOf(System.currentTimeMillis()))
            .event("notificacao")
            .data(msg)
            .retry(Duration.ofSeconds(3))
            .build());
  }

  @PostMapping("/sse/publicar")
  public void publicar(@RequestBody String mensagem) {
    sink.tryEmitNext(mensagem);  // broadcast para todos os clientes
  }
}
```

✦ *Sinks.many().multicast() — todos os clientes conectados recebem o evento simultaneamente, sem polling e sem alocação de thread por cliente*

SSE — Python (FastAPI) & Consumo no Browser

Python — sse-starlette

```python
# pip install sse-starlette
from sse_starlette.sse import EventSourceResponse
import asyncio, json
from datetime import datetime

async def sensor_generator(request):
    event_id = 0
    while True:
        if await request.is_disconnected():
            break
        event_id += 1
        yield {
            "id": str(event_id),
            "event": "temperatura",
            "data": json.dumps({
                "graus": 20 + event_id % 10,
                "ts": datetime.utcnow().isoformat()
            }),
            "retry": 3000
        }
        await asyncio.sleep(1)

@app.get("/api/sse/temperatura")
async def sse_endpoint(request: Request):
```

JavaScript — EventSource API

```javascript
// API nativa no browser
const source = new EventSource(
  "/api/sse/temperatura"
);

// Listener para tipo específico
source.addEventListener("temperatura", (e) => {
  const dados = JSON.parse(e.data);
  document.getElementById("temp").textContent
    = dados.graus + "°C";
  console.log("ID:", e.lastEventId);
});

source.onerror = (err) => {
  if (source.readyState === EventSource.CLOSED) {
    console.log("Conexão encerrada.");
  }
  // CONNECTING = reconectando automaticamente
};

// Encerrar quando necessário
// source.close();
```

WebSockets

Comunicação full-duplex persistente sobre TCP

WebSocket — Handshake e Frames

HTTP Upgrade (Handshake)

```
// Requisição do cliente
GET /ws/chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZQ==
Sec-WebSocket-Version: 13

// Resposta do servidor (101)
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9k...
```

Full-Duplex:	Cliente e servidor enviam mensagens a qualquer momento, simultaneamente
Frames:	Unidade mínima: texto (0x1), binário (0x2), ping (0x9), pong (0xA), close (0x8)
Latência mínima:	Sem overhead de headers HTTP por mensagem após o handshake inicial
Persistente:	Uma única conexão TCP sustenta milhares de mensagens bidirecionais

WebSocket vs SSE

Critério	SSE	WebSocket
Direção	Srv→Client	Bidirecional
Protocolo	HTTP	WS (TCP)
Browser	EventSource	WebSocket API
Reconexão	Automática	Manual
Overhead msg	HTTP headers	Mínimo
Ideal para	Feeds/Notif.	Chat/Jogos

WebSocket — Java Spring Boot

Java — TextWebSocketHandler

```java
@Configuration
@EnableWebSocket
public class WsConfig implements WebSocketConfigurer {
  @Override
  public void registerWebSocketHandlers(
                     WebSocketHandlerRegistry reg) {
    reg.addHandler(new ChatHandler(), "/ws/chat")
       .setAllowedOrigins("*");
  }
}

public class ChatHandler extends TextWebSocketHandler {
  private final List<WebSocketSession> sessions =
      new CopyOnWriteArrayList<>();

  @Override
  public void afterConnectionEstablished(
                       WebSocketSession session) {
    sessions.add(session);
  }

  @Override
  protected void handleTextMessage(
      WebSocketSession session, TextMessage msg) {
```

Python — FastAPI WebSocket

```python
from fastapi import FastAPI, WebSocket
from fastapi import WebSocketDisconnect

app = FastAPI()
conexoes = []

@app.websocket("/ws/chat/{usuario}")
async def chat(ws: WebSocket, usuario: str):
    await ws.accept()
    conexoes.append(ws)

    await broadcast({
        "tipo": "sistema",
        "msg": f"{usuario} entrou"
    })

    try:
        while True:
            dados = await ws.receive_json()
            await broadcast({
                "usuario": usuario,
                "texto": dados["texto"]
            })
    except WebSocketDisconnect:
```

gRPC Streaming

Streaming tipado sobre HTTP/2 com Protocol Buffers

gRPC — 4 Modalidades de Comunicação

Unary RPC

1 req → 1 resp

```
rpc Get(Request) returns (Response)
```

Caso de uso: API tradicional

✦ HTTP/2 multiplexado — múltiplos streams sobre uma conexão TCP

Client Streaming

stream → 1 resp

```
rpc Upload(stream Req) returns (Resp)
```

Caso de uso: Upload em lote

✦ HTTP/2 multiplexado — múltiplos streams sobre uma conexão TCP

Server Streaming

1 req → stream

```
rpc Monitor(Req) returns (stream Resp)
```

Caso de uso: Dashboards, feeds

✦ HTTP/2 multiplexado — múltiplos streams sobre uma conexão TCP

Bidirectional

stream → stream

```
rpc Chat(stream Req) returns (stream Resp)
```

Caso de uso: Chat, colaboração

✦ HTTP/2 multiplexado — múltiplos streams sobre uma conexão TCP

gRPC — Contrato .proto & Server Streaming em Java

sensor_service.proto

```
syntax = "proto3";
package sensor.v1;

service SensorService {
  // Unary
  rpc ObterLeitura(SensorRequest)
      returns (SensorResponse);

  // Server Streaming
  rpc MonitorarSensor(SensorRequest)
      returns (stream SensorResponse);

  // Client Streaming
  rpc EnviarLote(stream SensorRequest)
      returns (SummaryResponse);
}

message SensorRequest {
  string sensor_id = 1;
  int64 timestamp  = 2;
}
message SensorResponse {
  string sensor_id  = 1;
  double temperatura = 2;
```

Java — Server Streaming Implementation

```
@GrpcService
public class SensorServiceImpl
    extends SensorServiceGrpc.SensorServiceImplBase {

  @Override
  public void monitorarSensor(
      SensorRequest req,
      StreamObserver<SensorResponse> observer) {

    try {
      for (int i = 0; i < 100; i++) {
        if (Thread.interrupted()) break;

        observer.onNext(
            SensorResponse.newBuilder()
                .setSensorId(req.getSensorId())
                .setTemperatura(20 + Math.sin(i) * 5)
                .setUmidade(60 + Math.cos(i) * 10)
                .build()
        );
        Thread.sleep(1000);
      }
      observer.onCompleted();
    } catch (InterruptedException e) {
```

gRPC — Server & Client Streaming em Python

Python — Server Streaming

```
import grpc, time, math
import sensor_pb2, sensor_pb2_grpc

class SensorServicer(
        sensor_pb2_grpc.SensorServiceServicer):

  def MonitorarSensor(self, request, context):
    """yield = emite um item no stream"""
    i = 0
    while context.is_active():
      yield sensor_pb2.SensorResponse(
          sensor_id=request.sensor_id,
          temperatura=20.0 + math.sin(i) * 5.0,
          umidade=60.0 + math.cos(i) * 10.0,
      )
      time.sleep(1)
      i += 1

  def EnviarLote(self, request_iterator, context):
    """Client streaming: consome todos, retorna
resumo"""
    total = 0
    for req in request_iterator:
      total += 1
```

Python — Cliente gRPC

```
import grpc
import sensor_pb2 as pb
import sensor_pb2_grpc as pb_grpc

def cliente_server_streaming():
  with grpc.insecure_channel("localhost:50051") as
ch:
    stub = pb_grpc.SensorServiceStub(ch)
    req = pb.SensorRequest(sensor_id="S-01")

    # Itera sobre o stream do servidor
    for leitura in stub.MonitorarSensor(req):
      print(
        f"T={leitura.temperatura:.1f}°C "
        f"U={leitura.umidade:.1f}%"
      )


def cliente_client_streaming():
  def gerar_requisicoes():
    for i in range(50):
      yield pb.SensorRequest(
          sensor_id=f"S-{i:02d}"
      )
```

Reactive Streams

Backpressure, Flux e Project Reactor

Reactive Streams — Backpressure

O Problema Central

Sem backpressure, um Publisher rápido pode saturar a memória de um Subscriber lento — gerando OutOfMemoryError ou descarte silencioso de dados.

A Solução — Subscription.request(n)

O Subscriber solicita exatamente N elementos por vez. O Publisher não emite além do solicitado, respeitando a capacidade de processamento do consumidor.

As 4 Interfaces da Especificação Reactive Streams

`Publisher<T>`

Emite elementos.
subscribe(Subscriber)

`Subscriber<T>`

onSubscribe / onNext / onError / onComplete

`Subscription`

request(n) — solicita N elementos; cancel()

`Processor<T,R>`

Publisher + Subscriber: transforma T→R

Reactive Streams — Project Reactor (Java)

Flux e Backpressure Explícito

```
// Criação de Flux
Flux<Integer> numeros = Flux.range(1, 1000);
Flux<Long> ticks =
Flux.interval(Duration.ofSeconds(1));

// Pipeline com operadores
Flux.range(1, 20)
    .filter(n -> n % 2 == 0)
    .map(n -> "item-" + n)
    .take(5)
    .delayElements(Duration.ofMillis(100))
    .subscribe(System.out::println);

// Backpressure explícito
Flux.range(1, 1000)
    .onBackpressureBuffer(50)
    .subscribe(new BaseSubscriber<Integer>() {
      @Override
      protected void hookOnSubscribe(Subscription s) {
        request(10); // solicita apenas 10
inicialmente
      }
      @Override
      protected void hookOnNext(Integer value) {
```

Spring WebFlux — Pipeline HTTP Reativo

```
// SSE com pipeline reativo completo
@GetMapping(
  value = "/reactive/pipeline",
  produces = MediaType.TEXT_EVENT_STREAM_VALUE
)
public Flux<String> streamPipeline() {
  return Flux
    .interval(Duration.ofMillis(500))
    .take(100)
    .flatMap(
      id -> Mono
        .fromCallable(() -> buscarDados(id))
        .subscribeOn(Schedulers.boundedElastic()),
      3  // concorrência máxima = 3
    )
    .filter(Objects::nonNull)
    .onErrorContinue((err, val) ->
        log.warn("Erro ignorado: {}", err));
}

private String buscarDados(long id) {
  return String.format(
      {"id":%d,"val":%.1f}, id, Math.random()*100
  );
```

Reactive Streams — Python (Async Generators & Queue)

Async Generator (equivalente ao Flux)

```python
import asyncio
from typing import AsyncIterator

async def produtor(total: int,
                   delay: float = 0.1) ->
AsyncIterator[dict]:
    """
    Gerador assíncrono: backpressure natural —
    o produtor só avança quando o consumidor
    faz await no próximo item.
    """
    for i in range(total):
        await asyncio.sleep(delay)
        yield {"id": i, "valor": i ** 2}


async def consumidor():
    async for item in produtor(100, delay=0.05):
        # Consumidor 2x mais lento que o produtor
        await asyncio.sleep(0.1)
        print(f"Processado: {item}")

asyncio.run(consumidor())
```

asyncio.Queue — Backpressure Explícito

```python
import asyncio

async def pipeline_backpressure():
    """
    maxsize = buffer máximo.
    Produtor bloqueia se fila estiver cheia.
    """
    fila = asyncio.Queue(maxsize=10)

    async def produtor():
        for i in range(100):
            # Bloqueia se fila tiver 10 itens
            await fila.put(i)
            print(f"[P] {i}
(fila:{fila.qsize()}/10)")

    async def consumidor():
        while True:
            item = await fila.get()
            await asyncio.sleep(0.15) # lento
            fila.task_done()
            print(f"[C] {item}")
            if item == 99: break
```

Guia de Escolha — Quando Usar Cada Técnica

Dashboard / Monitoramento

→ Server-Sent Events (SSE)

Fluxo unidirecional, reconexão automática, compatível com proxies. Ideal para métricas e alertas em tempo real.

Exportação de Dados

→ HTTP Chunked Transfer

Sem protocolo especial. Ideal para dumps de BD, relatórios e arquivos de vários GBs sem buffering total.

Chat / Colaboração

→ WebSocket

Comunicação bidirecional com latência mínima. Essencial para chat, edição colaborativa e jogos multiplayer.

Notificações Simples

→ SSE ou Long Polling

SSE para sistemas modernos. Long Polling para compatibilidade com infraestrutura legada (HTTP/1.0).

Microserviços (interno)

→ gRPC Streaming

Tipagem forte (Protobuf), multiplexação HTTP/2, suporte multi-linguagem a partir do mesmo .proto.

Pipelines Assíncronos

→ Reactive Streams

Controle de backpressure, composição de operadores. Project Reactor (Java) ou async generators (Python).

Produção — Segurança, Infraestrutura e Testes

🔒 Segurança

- TLS obrigatório (wss:// e https://)
- Autenticação por JWT no handshake
- Rate limiting por IP/usuário
- Validação de origem (CORS)

⚡ Performance

- Modelo reativo (WebFlux, FastAPI)
- Thread pool dedicado para streams
- Heartbeats a cada 20–30s
- Monitorar conexões ativas (Micrometer)

🗄 Infraestrutura

- Nginx: proxy_buffering off (SSE)
- Nginx: Upgrade + Connection (WS)
- proxy_read_timeout 3600s
- Redis Pub/Sub para horizontal scaling

🧪 Testes

- pytest + httpx para SSE
- WebSocket mock com websockets lib
- TestWebSocketSession (Spring)
- Teste de backpressure com StepVerifier

CONCLUSÃO

Streaming de Dados para a Web

Chunked

Streaming simples sobre HTTP. Exportação progressiva.

WebSocket

Bidirecional em tempo real. Chat e jogos.

Long Polling

Compatibilidade legada. Baixo throughput.

gRPC

Tipagem forte + HTTP/2. Microserviços.

SSE

Unidirecional + reconexão automática. Dashboards.

Reactive

Backpressure e composição. Pipelines complexos.